

"What Was Decided" as a Standalone Accountability Question: The Substrate Carries Decisions as Inspectable Substrate Content in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 4, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one of the four accountability questions named in the source paper's path-retraceability commitment — the **"what was decided" question** — as a standalone architectural commitment with independent operational content, separable from the three companion questions ("by whom," "under what authority," "with what rationale") with which it composes into the integrated path-retraceability frame.

Abstract

The Coordination Knowledge Substrate (CKS) pattern's path-retraceability commitment names four accountability questions the substrate must answer architecturally for any decision the deployment makes: what was decided, by whom, under what authority, and with what rationale. A separate note establishes these questions as a structured set; the four together constitute the integrated commitment. This note formalizes one of the four — "what was decided" — as having independent architectural content. The question is the most fundamental of the four because the other three concern provenance metadata about a decision whose existence "what was decided" establishes: there is no writer to attribute, no authority context to identify, no rationale to capture if the decision itself is not substrate content. The note states the four operational components of the commitment (decisions are substrate content; addressable; persistent; inspectable), distinguishes the commitment from five adjacent patterns commonly conflated with it (decisions captured only in audit logs, decisions inferred from state changes, decisions held in cell-internal context, decisions recorded in external tracking systems, and decisions held only in LLM reasoning traces), enumerates the failure modes that violate it specifically, and provides an operational test for whether a system's decision capture is CKS-coherent in the architectural sense.

1. Why "what was decided" needs to be formalized as standalone

The CKS pattern's path-retraceability commitment names four accountability questions the substrate must answer architecturally for any decision the deployment makes: what was decided, by whom, under what authority, and with what rationale (§3.1 of the source paper). A separate derivation note establishes these four as a structured set and treats the integrated path-retraceability commitment in full operational detail; companion notes treat the six provenance fields and the substrate-only-paths property that complete the operational frame.

The integrated framing is correct as far as it goes, and this note does not contradict it. But it leaves a class of architectural content unspecified. The four questions are not symmetrical. "By whom," "under what authority," and "with what rationale" are about provenance — metadata that surrounds and contextualizes a decision. "What was decided" is about the decision itself: whether it exists as substrate content readers can locate, retrieve, and examine. Without it being answerable, the other three are moot — there is no writer to attribute, no authority context to identify, no rationale to capture if the decision itself is not substrate content.

The motivating cases are deployments where the substrate carries coordination state but does not record the specific decisions that produced it: substrates holding "current state" but not the decisions that updated it; substrates where decisions are recorded only in audit logs external to substrate; substrates where decisions are reconstructed from state diffs rather than recorded as decisions in their own right; substrates where multi-step decisions are committed as a single state change without the underlying decision steps preserved. Each pattern produces a system where "what is the case" is answerable from substrate but "what was decided" is not — the substrate carries the consequences of decisions without carrying the decisions.

A separate note formalizes the substrate-as-source-of-truth commitment (§11.3 of the source paper), which names "what was decided" as one of its authoritative categories. The source-of-truth note names the category; this note specifies the architectural commitment that makes the category answerable, in a form testable as a property in its own right and separable from how the deployment answers the other three questions.

2. The commitment, defined precisely

In the CKS pattern, the **"what was decided" commitment** is the architectural property that decisions produced by deployment activity exist as substrate content available for direct examination. The commitment has four operational components.

(a) Decisions are substrate content. For any decision the deployment makes — through cell execution under orchestration rules, direct human override, or rule authoring — the substrate contains the decision as a piece of substrate state. The decision is not metadata about substrate, not a reference to a decision held elsewhere, and not a record that a decision occurred. The decision itself is in the substrate, in whatever form the deployment chooses to represent it.

(b) Decisions are addressable. Each decision is addressable per the substrate layer's structure-with-addressability commitment. A reader can locate the decision as a discrete substrate object using the substrate's addressing conventions. Multiple decisions are individually addressable; aggregate decisions composed of multiple sub-decisions are addressable both as aggregates and as their

constituents, where the deployment's representation supports such decomposition.

(c) Decisions persist. Decisions remain in substrate per the substrate layer's persistence commitment, available for inspection across cell executions, sessions, and time. A decision made yesterday is retrievable today; a decision made years ago is retrievable now, modulo deletions taken by humans exercising the modify right under appropriate authority. Persistence is an architectural property of the substrate the decision is committed to, not a deployment-layer feature added on top of an ephemeral store.

(d) Decisions are inspectable. Humans exercising the inspect right can read decisions directly through the standard read operations the host environment provides, in inspectable form. The decision content preserves the decision's authoritative meaning rather than presenting only embedded numerical representations, vendor-specific encodings, or LLM-mediated curated summaries that depend on a model call to render readable.

The four components together define the commitment architecturally. A system that satisfies fewer than four cannot reliably answer "what was decided" for all deployment-produced decisions, regardless of how robustly the failed component is approximated by adjacent mechanisms.

3. What the commitment does NOT claim

Stating precisely what the commitment does not claim is what keeps the standalone treatment from drifting into something stronger than the source paper supports.

It does not claim that all events are recorded as decisions. Operational events such as system startup, host-layer configuration changes, monitoring alerts, and infrastructure transitions are not necessarily decisions in the architectural sense. The commitment is to decisions produced by deployment activity the architecture recognizes as decision-producing — cell executions under orchestration rules, direct human overrides, and rule authoring — not to all events in a deployment's history.

It does not claim that decisions are the only substrate content. Substrate also carries entities, relationships, conflicts, rationale, and other coordination state. Decisions are one category of substrate content; the commitment specifies that this category is architecturally complete for decisions the deployment produces.

It does not claim that decisions have any particular schema. The substrate's representation of decision content is a deployment choice. Decision content can be structured in any form the deployment requires, provided the structure satisfies the addressability commitment in §2(b) and the inspectable-form requirement in §2(d).

It does not claim that decisions are interpreted automatically. Reading a decision returns its substrate content as the substrate carries it. Making sense of the decision — what it implies, how it relates to other decisions, what its consequences are — is the reader's task. The commitment is to availability; interpretation is operational, not architectural.

It does not specify retention duration. Decisions persist as substrate content for as long as the substrate holds them. How long any deployment retains decisions is a deployment concern; the commitment is that decisions are not ephemeral within the substrate's existence.

It does not require any particular human-readable language or format. Decision content may be in any format the deployment chooses, provided humans can inspect it through the host environment's standard read operations and the decision's authoritative meaning is preserved in the form the substrate carries.

4. What the commitment is NOT

Five adjacent patterns are commonly conflated with the "what was decided" commitment. Each is a real and reasonable approach in some other architecture; naming what the commitment is not is what prevents the misreading.

Not decisions captured in audit logs. Audit logs record events that occurred — including decisions — for after-the-fact review at a different architectural layer. The commitment is that decisions are substrate content, not audit-log content. Audit logs may complement substrate as deployment-layer features, but a substrate where decisions exist only in audit logs fails the commitment because the substrate does not carry the decisions architecturally. Implementations where "the substrate is the audit log" violate the architectural distinction between substrate (architectural state with governance commitments) and audit log (operational record without those commitments).

Not decisions inferred from state changes. Some patterns reconstruct decisions by comparing substrate state at different points in time — what changed between time T and time T+1 implies a decision was made. The commitment is to decisions being explicit substrate content, not inferred from state diffs. State diffs may be useful operational information but are not substitutes for explicit decision records: readers must reconstruct decisions rather than read them, and multiple decision sequences can produce the same state diff without disambiguation.

Not decisions held in cell-internal context. During a cell execution, the cell may form intermediate decisions that contribute to its eventual substrate write. Per the cell-internal-vs.-substrate-state distinction, cell-internal state is released when execution completes; only the cell's substrate writes persist. The commitment is to the cell's commit-time decisions being substrate content; intermediate decisions internal to the cell's execution are not substrate-bound and do not need to be. What the commitment requires is that whatever decisions the cell commits to substrate are substrate content.

Not decisions recorded in external tracking systems. Some deployments track decisions in external systems — project management tools, decision-logging applications, governance dashboards. These may record decisions, but the commitment is that the substrate carries decisions, not external systems. External tracking can complement substrate but cannot substitute for it. A substrate that does not carry decisions, with decisions only in external systems, fails the commitment because the substrate cannot answer "what was decided" from substrate alone — which violates the substrate-only-paths property the integrated path-retraceability frame depends on.

Not decisions held in LLM reasoning traces. A pattern increasingly common in agent-framework deployments is for decisions to exist only as tokens in an LLM's reasoning output — chain-of-thought traces, tool-use planning sequences, model-internal deliberations — with the substrate carrying only the action the reasoning ultimately produced. The reasoning trace exhibits decision content but is not committed to substrate; it is transient context within a model call. Reasoning traces are neither persistent (the trace is released when the call completes), nor addressable (no substrate object identifier), nor reliably inspectable (the trace exists only in model output, accessible only if the host

captures it as a deployment-layer feature). A substrate where the only record of "why this action was taken" lives in model output fails the commitment, regardless of the trace's detail at the time of production.

5. Why "what was decided" is load-bearing

The commitment is load-bearing for several CKS commitments downstream.

Source-of-truth. The substrate is authoritative for "what was decided" as one of its categories (§11.3). Without the question being architecturally answerable, the source-of-truth commitment fails for that category — the substrate may be authoritative for current state but not for the decisions that established it.

Human-governed. Humans cannot exercise meaningful governance over decisions they cannot inspect. The commitment is what makes decisions inspectable architecturally; without it, governance over decisions becomes nominal — humans can override the system's current state but cannot examine the decisions whose effects they are overriding.

Conflict as first-class. Conflicts between decisions require both decisions to be substrate content for the conflict to be addressable through the relationship-metadata fields the conflict-preservation commitment depends on. Without "what was decided" being answered, conflicts cannot be first-class because the conflicting decisions are not substrate objects.

Substrate-only paths. The integrated path-retraceability frame commits the substrate to supporting paths reconstructible from substrate alone. Paths through substrate content require decisions on the path to be substrate content; paths cannot be reconstructed from substrate alone if the decisions on the path live elsewhere.

The other three accountability questions. "By whom" attributes a decision to a writer; "under what authority" identifies the authority context for a decision; "with what rationale" captures why a decision was made. All three depend on the decision itself existing as substrate content. The other three questions presuppose the answer to "what was decided" architecturally.

6. Failure modes that violate the commitment

A system can fail the "what was decided" commitment specifically, in ways distinct from failures of the other accountability questions or the broader path-retraceability frame. Nine failure modes name the most common.

(a) Decisions in audit logs only. The deployment records decisions in audit logs external to substrate; substrate carries only the resulting state. Substrate cannot answer "what was decided"; readers must consult audit logs whose architectural status differs from substrate.

(b) State-tracking without decision-recording. The deployment tracks substrate state changes over time but does not explicitly record the decisions that produced them. Readers must infer decisions from state diffs, which does not disambiguate decision sequences that produce the same diff.

(c) Decisions in cell-internal context that escape to logs. Cells form decisions during execution; the decisions are captured in execution logs (cell-internal context not properly committed, or runtime logs

of the cell's execution) rather than in substrate writes. The decisions are recorded somewhere, but the somewhere is not substrate.

(d) Summarized decisions. The deployment commits summaries of multi-step decisions to substrate, with the underlying steps not preserved. Readers see what was summarized but cannot examine the contributing decisions, and the summary's faithfulness is not architecturally guaranteed.

(e) External-tracking-system decisions. The deployment uses external tools to track decisions; substrate references those tools but does not carry the decisions itself. The substrate-only-paths property is broken because answering "what was decided" requires a system call outside substrate.

(f) Implicit decisions. Some decisions are made by the architecture's automated processes (rule applications, cell executions under rules) but are not recorded as distinct substrate content. The deployment treats the resulting substrate state as the only record; the decisions are implicit and cannot be examined.

(g) Pseudonymous-state decisions. Decisions are recorded in substrate but in forms that do not preserve their authoritative meaning — heavily abbreviated codes, identifiers without explanatory content, or embedded representations only. Readers can locate the substrate content but cannot read what was decided in inspectable form.

(h) Cumulative-state-only patterns. The deployment maintains only the current substrate state, with prior decisions overwritten or lost as state evolves. Readers can see current state but cannot retrieve prior decisions; "what was decided" is answerable for current state but not historically.

(i) LLM-reasoning-trace decisions. Decisions exist as tokens in an LLM's reasoning output during a model call but are not committed to substrate; the substrate carries only the actions the reasoning produced. The reasoning trace exhibits decision content but does not persist, is not addressable, and is not directly inspectable as substrate. This failure is increasingly common in agent-framework deployments where the framework treats the model's chain-of-thought as the decision record.

A system exhibiting any of (a)–(i) does not satisfy the commitment in the architectural sense, even if it provides decision-related capture in some other form.

7. Operational test

A system satisfies the "what was decided" commitment if and only if all of the following are true at all times during the substrate's existence:

1. For every decision produced by deployment activity recognized as decision-producing (cell executions under orchestration rules, direct human overrides, rule authoring), the substrate contains the decision as substrate content.
2. Decisions are addressable per the substrate layer's structure-with-addressability commitment; readers can locate decisions as discrete substrate objects.
3. Decisions persist per the substrate layer's persistence commitment; decisions are retrievable across cell executions, sessions, and time, modulo human-authored deletions taken under the modify right.
4. Decisions are inspectable per the host environment's standard read operations; humans can read decisions directly in inspectable form, without LLM intermediation as a precondition and without reverse-engineering compiled or embedded representations.
5. The substrate content held under (1)–(4) is the decisions themselves, not substitutes — not audit-log

entries about decisions, not state diffs implying decisions, not summary records describing decisions, not pointers to decisions held in external systems, and not LLM reasoning outputs that exhibit decisions without committing them.

A system that fails any of (1)–(5) does not satisfy the commitment in the architectural sense, even if it provides decision capture in some adjacent form.

8. Why naming "what was decided" as standalone matters

Implementations under organizational pressure to integrate with audit infrastructure, project management tools, or external decision-logging systems consistently drift toward decisions held outside substrate. The drift is steady because external systems are operationally familiar, well-tooled, and organizationally established for decision tracking; agent frameworks add a parallel pressure to treat LLM reasoning as the record of why an action was taken. The architectural commitment to substrate-carrying-decisions is what each external integration and each agent-framework convention erodes when treated as a substitute rather than a complement.

Implementations that drift produce systems where the substrate appears to function but cannot answer "what was decided" from substrate alone. The downstream consequences manifest as path-retraceability failures (decisions cannot be traced through substrate paths because the decisions are not in substrate), source-of-truth failures (the substrate is authoritative for current state but not for the decisions that established it), and substrate-only-paths failures (paths require external systems or transient model outputs).

Naming "what was decided" as a standalone architectural commitment gives downstream implementers a precise specification of what the architectural commitment to decision-capture requires, separable from how the other three accountability questions are answered. Subsequent notes formalize "by whom," "under what authority," and "with what rationale" as standalone questions; another formalizes the six provenance fields per substrate write; another formalizes the substrate-only-paths property. Together with this note, they give the full operational decomposition of the integrated path-retraceability commitment.

Subsequent work that adopts, extends, or argues against the CKS path-retraceability commitment should use "what was decided" in the sense formalized here. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). "What Was Decided" as a Standalone Accountability Question: The Substrate Carries Decisions as Inspectable Substrate Content in the Coordination Knowledge Substrate Pattern. May 4, 2026. ORCID: 0009-0004-8065-3235.